

Retrieval & Summarization

Dr.Madhusudhanan

<https://www.linkedin.com/in/drmadhub/>

Retrieval

Retrieval is the process of **finding and selecting the most relevant pieces of text (documents or chunks)** from a large collection based on a user query.

In simple terms:

Retrieval = Search + Select relevant information

Why Retrieval is Needed

- Large document collections cannot be processed entirely for every query.
- Retrieval helps the system focus only on **useful and relevant content**.
- Improves **speed, accuracy, and relevance** of downstream tasks like QA or summarization.

How Retrieval Works

Step 1: Data Preparation

- A long document is split into smaller chunks (sentences, paragraphs, or sections).
- Each chunk is stored in a database (text + metadata).

Step 2: Query Input

User asks:

“How does AI help in medical diagnosis?”

How Retrieval Works

Step 3: Matching

- The system compares the query with stored chunks.
- It identifies chunks related to:
 - AI
 - Healthcare
 - Diagnosis

Step 4: Retrieval Output

- Only the **top relevant chunks** are returned.
- Example retrieved chunks:
 - “AI helps doctors detect diseases from medical images...”
 - “Machine learning models assist in early diagnosis...”

Types of Retrieval

1. Keyword-Based Retrieval

- Matches exact words (e.g., TF-IDF, BM25).
- Example: Searches for the word “*diagnosis*”.

2. Semantic Retrieval

- Matches **meaning**, not exact words.
- Example:
 - Query: “*disease detection*”
 - Retrieves text containing “*medical diagnosis*”.

3. Hybrid Retrieval

- Combines keyword + semantic search.
- Improves accuracy and coverage.

Vector Store Retriever

- Vector Store Retrievers use **embedding-based similarity search** to find the most relevant chunks from a document collection.
- Uses **embeddings + similarity search**
- Backbone of most LangChain applications

Popular vector stores:

- FAISS
- Chroma
- Pinecone

Use when:

- You want **semantic search**
- Building QA, chatbots, or RAG systems

Vector Store Retriever - Mathematical Representation

1. Embedding each chunk:

After chunking, each text chunk is converted into a **vector representation** using an embedding function.

This allows semantic comparison and similarity-based retrieval.

$$\vec{v}_i = f(C_i)$$

where:

- $C_i = i^{th}$ chunk of text
- $f(\cdot)$ = embedding function (e.g., BERT, OpenAI embeddings)
- $\vec{v}_i$ = vector representation of chunk C_i

Vector Store Retriever - Mathematical Representation

2. Embedding the query:

Before retrieving relevant chunks, the user's query is converted into a **vector embedding**.

This allows the system to perform **semantic similarity search** between the query and the document chunks.

$$\vec{q} = f(Q)$$

where:

- Q = user query
- $\vec{q}$ = query embedding

Vector Store Retriever - Mathematical Representation

3. Similarity search:

Once both the **query** and **chunks** are converted into embeddings, a similarity search is performed to identify the most relevant chunks.

How It Works:

- Compute a **similarity score** between the query vector and each chunk vector
- The most common method is **cosine similarity**, which measures how close two vectors are in meaning
- Higher similarity scores indicate more semantically relevant chunks

Vector Store Retriever - Mathematical Representation

3. Similarity search:

Key Concepts:

- **Query vector:** Represents the meaning of the user's query
- **Chunk vector:** Represents the meaning of a text chunk
- **Similarity score:** Quantifies how closely the query and chunk relate

$$\text{score}(Q, C_i) = \cos(\vec{q}, \vec{v}_i) = \frac{\vec{q} \cdot \vec{v}_i}{\|\vec{q}\| \|\vec{v}_i\|}$$

where:

- $\cos(\vec{q}, \vec{v}_i)$ = cosine similarity between query and chunk
- $\cdot$ = dot product
- $\|\vec{q}\|, \|\vec{v}_i\|$ = Euclidean norm of vectors

Vector Store Retriever - Mathematical Representation

4. Retrieval step:

After computing similarity scores between the query and all chunks, the system selects the **most relevant chunk(s)** for use in downstream tasks like QA, summarization, or chat responses.

How It Works:

- Compare the query embedding with embeddings of all chunks
- Identify the chunk(s) with the **highest similarity score**
- These selected chunks are considered the **most relevant** to the query

Vector Store Retriever - Mathematical Representation

4. Retrieval step:

Key Concept:

- **Retrieved chunk(s):** The text segment(s) that best match the user's query semantically
- This step ensures that only relevant information is passed to the model

$$C^* = \arg \max_{C_i} \text{score}(Q, C_i)$$

where:

- C^* = most relevant chunk(s) retrieved

BM25 Retriever

BM25 is a **keyword-based retrieval model** widely used for **exact keyword matching**.

- No embeddings are required
- Lightweight and fast, ideal for large textual datasets

How It Works (Conceptually)

- Each query term is compared with documents in the corpus
- BM25 scores each document based on:
 - Frequency of the query term in the document
 - Document length relative to the average document length in the corpus
 - Overall occurrence of the term across all documents
- Hyperparameters control **term frequency saturation** and **length normalization** to improve retrieval accuracy

BM25 Retriever - Mathematical Representation

The BM25 relevance score of a document D for a query Q is:

$$\text{score}(D, Q) = \sum_{q_i \in Q} \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \cdot \log \frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}$$

where:

- q_i = the i^{th} query term in Q
- $f(q_i, D)$ = frequency of term q_i in document D
- $|D|$ = length of document D in words
- avgdl = average document length in the corpus
- N = total number of documents
- $n(q_i)$ = number of documents containing term q_i
- k_1, b = hyperparameters controlling term frequency saturation and length normalization

Rank Fusion

Rank Fusion Methods

Rank Fusion is the process of **combining multiple ranked lists of documents**, produced by different retrieval systems, into a **single unified ranking**.

- Each retriever provides its own ranked output.
- Rank fusion merges these outputs to improve **robustness, recall, and relevance**.

Why Rank Fusion is Needed

- Different retrievers capture **different signals**:
 - Keyword match
 - Semantic similarity
 - Metadata relevance
- A single retriever may miss important results.
- Rank fusion leverages **complementary strengths** without replacing retrievers.

Categories of Rank Fusion Methods

1. **Rank-Based Fusion Methods:** Combine results using only the **rank positions** of documents from multiple retrievers, ignoring raw scores.
2. **Score-Based Fusion Methods:** Merge retrieval results by **combining relevance scores** from different retrievers, often after score normalization.
3. **Weighted Fusion Methods:** Fuse rankings or scores by assigning **different importance (weights)** to each retriever based on trust or performance.

Reciprocal Rank Fusion (RRF)

Reciprocal Rank Fusion is a **rank-based fusion method** that combines results from multiple retrievers using only their **rank positions**, not raw scores. This makes RRF robust when retrievers produce scores on different scales.

$$\text{RRF}(D) = \sum_{j=1}^M \frac{1}{k + \text{rank}_j(D)}$$

where

- $\text{rank}_j(D)$ = rank of document D given by retriever R_j
- k = smoothing constant (typically $k \approx 60$)
- M = number of retrievers

Documents appearing higher in multiple ranked lists receive larger RRF scores and are ranked higher overall.

Score-Based Rank Fusion

- Score-based rank fusion combines the outputs of multiple retrievers by **aggregating their relevance scores**.
- Each retriever evaluates the same document independently, and the final relevance is computed as a **weighted sum** of individual scores.

$$\text{Score}_{\text{fused}}(D) = \sum_{j=1}^M w_j \cdot \text{Score}_j(D)$$

where:

- $\text{Score}_j(D)$ = score given by retriever R_j
- w_j = weight assigned to retriever R_j
- M = total number of retrievers

Documents are ranked based on $\text{Score}_{\text{fused}}(D)$.

Re-Ranking

Re-Ranking

Re-Ranking is the process of **reordering a small set of retrieved documents** using deeper and more accurate relevance evaluation before passing them to a language model.

- Applied **after retrieval (and rank fusion, if used)**
- Operates on **top-k candidates**, not the full corpus
- Focuses on **precision rather than speed**

Why Re-Ranking is Needed

- Initial retrieval is fast but approximate
- Top retrieved results may not be optimally ordered
- Re-ranking improves:
 - Relevance
 - Answer quality
 - Context alignment

Cross-Encoder Re-Ranker

- **Most popular re-ranking approach** in retrieval pipelines
- Evaluates **query and document together** rather than separately
- Produces **highly accurate relevance scores**

A cross-encoder model computes:

$$\text{Score}(Q, D) = f_{\theta}([Q; D])$$

where:

- Q = user query
- D = document or chunk
- $[Q; D]$ = concatenation of query and document text
- f_{θ} = cross-encoder neural model producing a **relevance score**

Documents are ranked in descending order of $\text{Score}(Q, D)$

Other Re-Rankers

Cohere Re-Ranker:

- Cohere Re-Rankers are **neural re-ranking models** that assess the relevance of a query-document pair to improve retrieval results.
- They refine initial candidate lists from other retrievers to produce **better-ranked, more relevant results**.

LLM-Based Re-Ranker:

- LLM-Based Re-Rankers leverage **large language models** to evaluate the relevance of query-document pairs.
- They provide **context-aware, semantic rankings** by understanding the meaning of both the query and the document.

Summarization

Introduction to Text Summarization

Text Summarization is the process of generating a concise and coherent version of a longer text document.

It preserves key information while reducing length.

It is a core task in **Natural Language Processing (NLP)**.

Applications:

- News aggregation
- Research paper summarization
- Legal document summarization
- Meeting transcript summarization
- Chat/email summarization

Need for Text Summarization

- Exponential growth of digital textual data
- Information overload problem
- Time efficiency in decision-making
- Helps in quick knowledge extraction
- Essential for business intelligence and research

*If original document length = n
Summary length = k , where $k \ll n$*

Goal:

- Maximize Information Retention
- Minimize Redundancy

Types of Summarization

Extractive Summarization

- Selects important sentences directly from original text
- No modification of words
- Based on scoring and ranking mechanisms

Abstractive Summarization

- Generates new sentences
- May paraphrase original content
- Uses deep learning & language models

Abstractive Summarization – Working Principle

Treated as a Sequence-to-Sequence (Seq2Seq) Problem: Abstractive summarization is formulated as a sequence transformation task where an input text sequence is converted into a shorter output sequence (summary).

Encoder–Decoder Architecture: The encoder reads and compresses the input text into a contextual representation, and the decoder generates the summary from this representation.

Abstractive Summarization – Working Principle

Attention Mechanism: Allows the model to focus on important parts of the input text while generating each word of the summary, improving relevance and coherence.

Transformer-Based Models: Modern models use self-attention mechanisms instead of recurrent networks, enabling better handling of long-range dependencies and parallel processing.

Abstractive Summarization – Working Principle

Objective

Given input sequence **X**, generate summary sequence **Y**.

The model learns the conditional probability:

$$P(Y | X)$$

which represents the probability of generating summary **Y** given input document **X**.

BART – Overview and Architecture

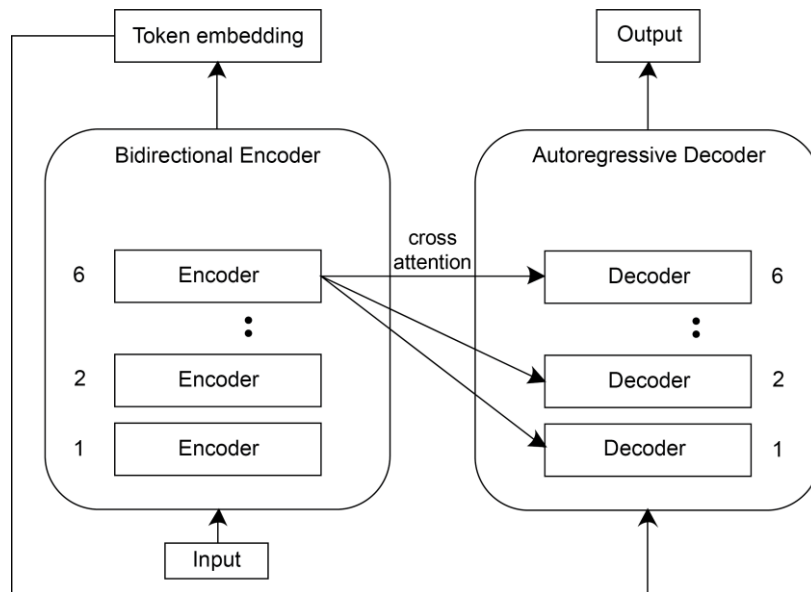
BART (Bidirectional and Auto-Regressive Transformers)

- Developed by Meta AI for sequence-to-sequence text generation tasks.
- Combines **Bidirectional Encoder (like BERT)** and **Autoregressive Decoder (like GPT)**.
- Built using the standard **Transformer encoder–decoder architecture**.
- Designed for tasks such as:
 - Text summarization
 - Text generation
 - Translation
 - Question answering

Key Idea

BART is trained as a **denoising autoencoder**, where corrupted input text is reconstructed into the original clean text.

BART – Overview and Architecture



BART – Working Principle and Training

Denoising Pretraining Strategy

- The input text is intentionally corrupted using:
 - Token masking
 - Token deletion
 - Sentence permutation
 - Text infilling
- The model learns to reconstruct the original text from noisy input.

Why BART Works Well for Summarization

- Learns strong contextual representations from bidirectional encoder.
- Decoder generates fluent summaries autoregressively.
- Handles long-range dependencies effectively.

T5 – Overview and Architecture

T5 (Text-to-Text Transfer Transformer)

- Developed by Google Research as a unified framework for all NLP tasks.
- Treats every NLP task as a **text-to-text problem**.
- Based on the standard **Transformer encoder–decoder architecture**.
- Input and output are always formatted as text strings.

Core Idea

Instead of designing task-specific architectures, T5 converts tasks like:

- Summarization
- Translation
- Classification
- Question answering

T5 – Overview and Architecture

